CO326

Edge AI + Industrial IoT Mini Project

Room Temperature Anomaly Detection

Project 18 - Environmental & Smart Systems

Project Title	Room Temperature Anomaly Detection
Module	CO326
Submission Date	May 2026
GitHub Repository	https://github.com/cepdnaclk/e20-co326-Room-Temperature-Anomaly-Detection

Group Members

Member 1 (E/20/024)	Hardware & ESP32 Firmware
Member 2 (E/20/346)	Edge AI & Python
Member 3 (E/20/350)	Documentation & GitHub
Member 4 (E/20/378)	Dashboard & Node-RED

1. Introduction

This report presents the design and implementation of a Room Temperature Anomaly Detection system developed as part of the CO326 module. The system monitors ambient temperature and humidity in real time, applies edge-based AI logic to detect anomalous conditions, and provides visual and physical alerts when anomalies are identified.

Temperature anomalies in indoor environments - such as sudden spikes, sustained overheating, or unusual drift patterns, can signal equipment failures, safety hazards, or HVAC malfunctions. By deploying intelligence at the edge (on a local PC rather than a cloud server), the system can respond to anomalies with minimal latency and without dependence on internet connectivity.

1.1 Objectives

- Acquire real-time temperature and humidity data using a DHT22 sensor and ESP32-S3 microcontroller.
- Transmit sensor data to a local MQTT broker over WiFi.
- Apply edge-based anomaly detection in Python to identify temperature spikes, threshold violations, and drift patterns.
- Visualize live sensor data and alerts on a Node-RED dashboard.
- Trigger a physical relay and buzzer when an anomaly is detected.
- Maintain all code and documentation on GitHub.

1.2 Scope

The project covers both hardware implementation (ESP32-S3, DHT22, relay, buzzer, LEDs) and software implementation (Python edge AI, MQTT communication, Node-RED dashboard, Docker-based deployment). Both real hardware data and software simulation are supported.

2. System Architecture

The system follows a layered Edge AI IoT architecture. Data flows from the physical sensor layer through communication middleware to a local processing and visualization layer.

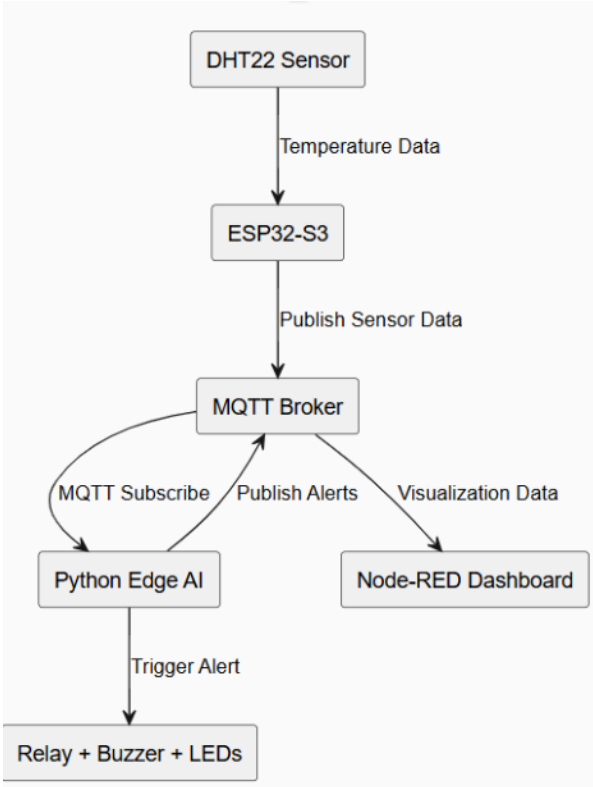

Figure 1: Overall System Architecture Diagram

2.1 Architecture Overview

The pipeline consists of five main components:

- DHT22 Sensor - measures temperature (°C) and humidity (%) every 2 seconds.
- ESP32-S3 Microcontroller - reads sensor data, formats it as JSON, and publishes to the MQTT broker over WiFi.
- Mosquitto MQTT Broker (Docker) - receives published messages and routes them to all subscribers.
- Python Edge AI (edge_ai.py) - subscribes to sensor data, runs anomaly detection logic, and publishes alert messages.
- Node-RED Dashboard (Docker) - subscribes to both sensor data and alerts, visualizing them in real time with gauges, charts, and status indicators.

Layer	Component	Technology	Role
Sensing	DHT22 + ESP32-S3	Arduino/C++	Data acquisition
Communication	Mosquitto MQTT	Docker	Message broker
Processing	edge_ai.py	Python	Anomaly detection
Visualization	Node-RED	Docker	Dashboard & alerts
Physical Output	Relay + Buzzer + LEDs	Hardware	Physical alert response

Table 1: System Components and Technologies

2.2 MQTT Topic Structure

All communication follows a structured MQTT topic hierarchy:

```
sensors/group11/temperature/data    ← ESP32 publishes sensor readings
alerts/group11/temperature/status   ← Edge AI publishes anomaly alerts
```

Each sensor message is a JSON payload with the following structure:

```
{ "group_id": "group11", "temperature": 28.5, "humidity": 62.3, "timestamp": 1234567.89 }
```

(Note: 11 is the project number)

3. Hardware Design

3.1 Components

Component	Model	Quantity	Purpose
Microcontroller	ESP32-S3	1	WiFi + sensor control
Temperature/Humidity Sensor	DHT22	1	Environmental sensing
Relay Module	5V Single Channel	1	Control buzzer/fan
Buzzer	5V Active Buzzer	1	Audible anomaly alert
LED (Green)	5mm LED	1	Normal status indicator
LED (Red)	5mm LED	1	Anomaly status indicator
Resistors	330Ω	2	LED current limiting
Breadboard + Jumper Wires	-	1 set	Prototyping connections

Table 2: Hardware Components

3.2 Circuit Wiring

ESP32-S3 Pin	Component	Notes
GPIO4	DHT22 DATA	10kΩ pull-up to 3.3V
3.3V	DHT22 VCC	Power supply
GND	DHT22 GND	Common ground
GPIO5	Relay IN	Active HIGH trigger
5V	Relay VCC	Relay power
GPIO2	Green LED (+330Ω)	Normal status
GPIO38	Red LED (+330Ω)	Anomaly alert
Relay COM/NO	Buzzer	Normally open - buzzer activates on anomaly

Table 3: Pin Connections

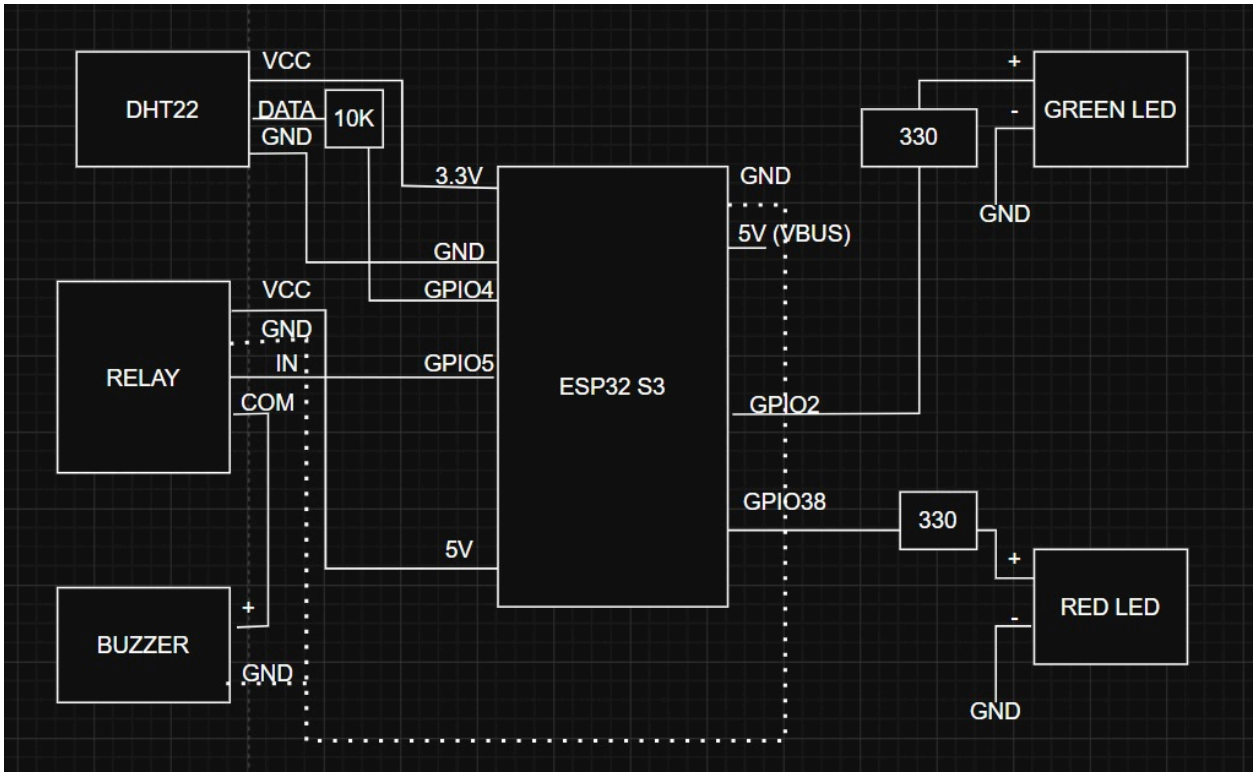

Figure 2: Hardware Circuit Diagram / Wiring Photo

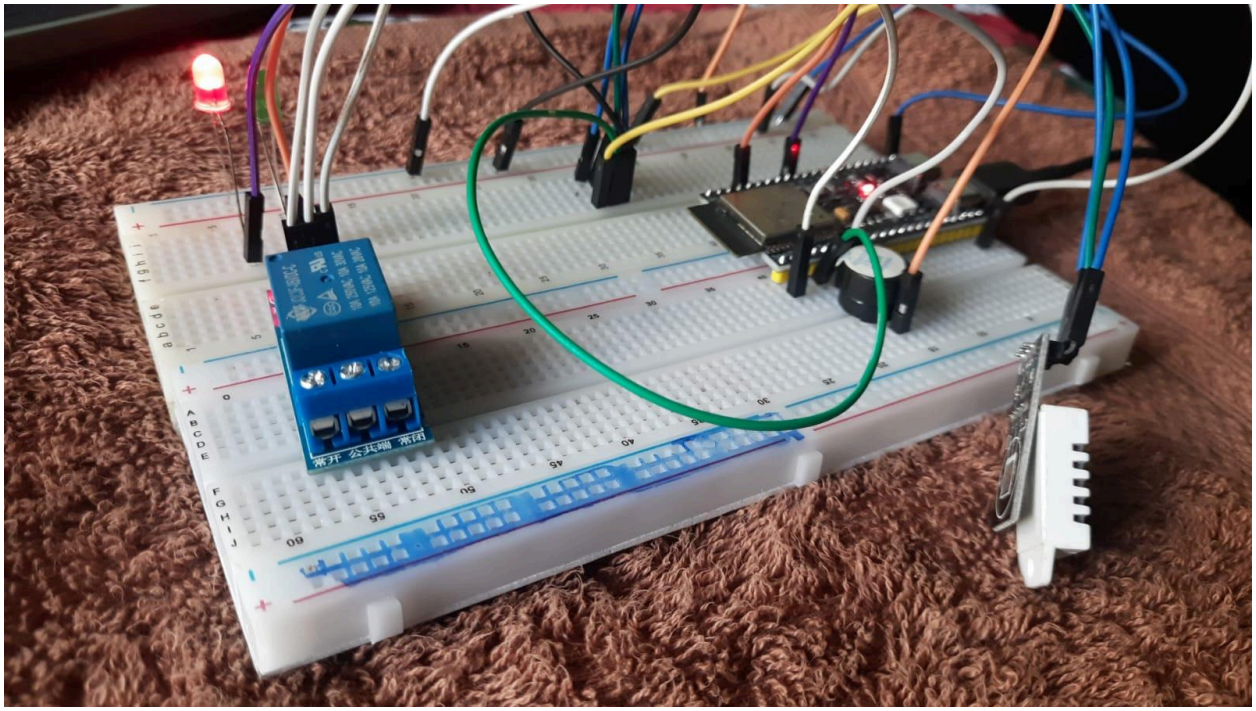

Figure 3: Hardware Circuit

4. Software Implementation

4.1 ESP32-S3 Firmware

The firmware is developed using PlatformIO on VS Code with the Arduino framework. It uses three libraries: DHTesp for sensor reading, PubSubClient for MQTT communication, and ArduinoJson for JSON serialization.

The firmware performs the following operations in the main loop:

- Reads temperature and humidity from DHT22 every 2 seconds.
- Formats readings into a JSON payload with group ID and timestamp.
- Publishes the payload to the MQTT broker over WiFi.
- Listens for alert messages from the Edge AI and activates the relay, red LED, and buzzer on ANOMALY status; green LED on NORMAL status.

```
==================================================
   ESP32-S3 Room Temperature Anomaly Detection
==================================================
   High threshold : > 32.0 C
   Low  threshold : < 15.0 C
   Spike limit    : > 5.0 C jump
==================================================
   Starting sensor readings...
==================================================


--------------------------------------------------
[00:10] Reading #1
  Temp     : 22.5 C
  Humidity : 55.0 %
--------------------------------------------------
  STATUS   : NORMAL
  METHOD   : NORMAL
  REASON   : Within safe range (Comfortable morning)
  ACTION   : Green LED ON | Relay OFF | Buzzer OFF
--------------------------------------------------
```

```
--------------------------------------------------
[05:10] Reading #31
  Temp     : 42.0 C
  Humidity : 80.0 %
-------------|------------------------------------
  STATUS   : *** ANOMALY DETECTED ***
  METHOD   : THRESHOLD
  REASON   : Temperature too HIGH! (OVERHEATING! AC failure!)
  ACTION   : Red LED ON | Relay ON | Buzzer ON
--------------------------------------------------
```

```
--------------------------------------------------
[08:20] Reading #50
  Temp     : 10.5 C
  Humidity : 30.0 %
--------------------------------------------------
  STATUS   : *** ANOMALY DETECTED ***
  METHOD   : THRESHOLD
  REASON   : Temperature too LOW! (TOO COLD! Heating failure!)
  ACTION   : Red LED ON | Relay ON | Buzzer ON
--------------------------------------------------

--------------------------------------------------
[08:40] Reading #51
  Temp     : 22.5 C
  Humidity : 55.0 %
--------------------------------------------------
  STATUS   : NORMAL
  METHOD   : NORMAL
  REASON   : Within safe range (Comfortable morning)
  ACTION   : Green LED ON | Relay OFF | Buzzer OFF
--------------------------------------------------
```

Figure 4: Serial Monitor Output — Successful Sensor Readings and MQTT Publish

4.2 Docker Infrastructure

The broker and dashboard are deployed using Docker Compose, providing a reproducible and portable environment. Two containers are used:

- eclipse-mosquitto:2.0 - MQTT broker listening on port 1883 across all interfaces (0.0.0.0).
- nodered/node-red:latest - Dashboard accessible at http://localhost:1880.

The Mosquitto configuration uses allow_anonymous true for simplicity in the lab environment. The broker binds to 0.0.0.0:1883 to accept connections from the ESP32 on the local WiFi network, not just from localhost.

4.3 Edge AI - Anomaly Detection

The anomaly detection logic runs in edge_ai.py on the local PC. It subscribes to the sensor data topic, processes each incoming reading, and publishes an alert if an anomaly is detected. Three detection rules are applied:

Rule	Condition	Rationale
Hard Threshold	Temperature > 32.0°C	Indicates overheating above acceptable room temperature
Spike Detection	Current reading deviates > 5°C from recent 9-point moving average	Catches sudden anomalous jumps
Low Temperature	Temperature < 18.0°C	Detects abnormally cold conditions

Table 4: Anomaly Detection Rules

When an anomaly is detected, the alert payload includes the status field set to ANOMALY along with the specific reasons and the triggering temperature value. The ESP32 subscribes to the alert topic and activates the relay and red LED upon receiving ANOMALY status.

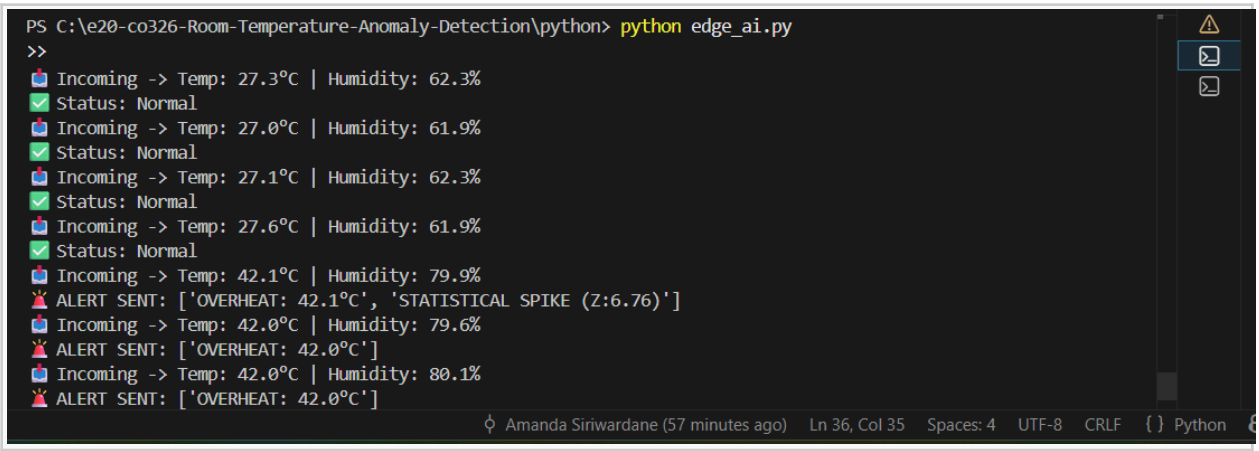

Figure 5: Edge AI Terminal Output — Normal Readings and Anomaly Detection

4.4 Node-RED Dashboard

The Node-RED flow subscribes to both the sensor data topic and the alert topic. A function node parses the incoming JSON and extracts individual fields, routing them to the appropriate dashboard widgets.

The dashboard includes the following widgets:

- Temperature Gauge - real-time needle gauge with colour zones (green 0–25°C, yellow 25–35°C, red 35–50°C).
- Temperature History Chart - rolling line chart showing the last 100 readings with timestamps.
- Temperature and Humidity Text Displays - live numeric readouts.
- Alert Status Text - displays the latest alert payload including anomaly reasons.

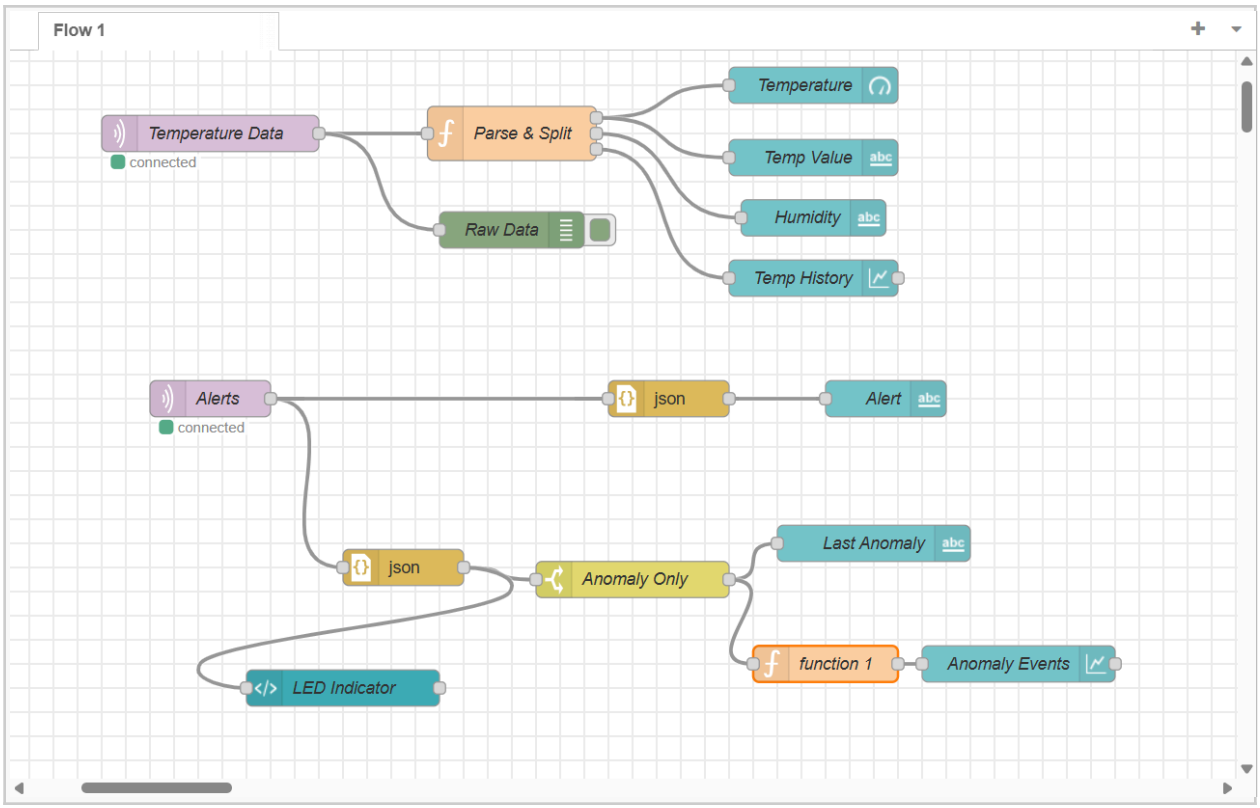

Figure 6: Node-RED Flow Editor — Complete Flow Diagram

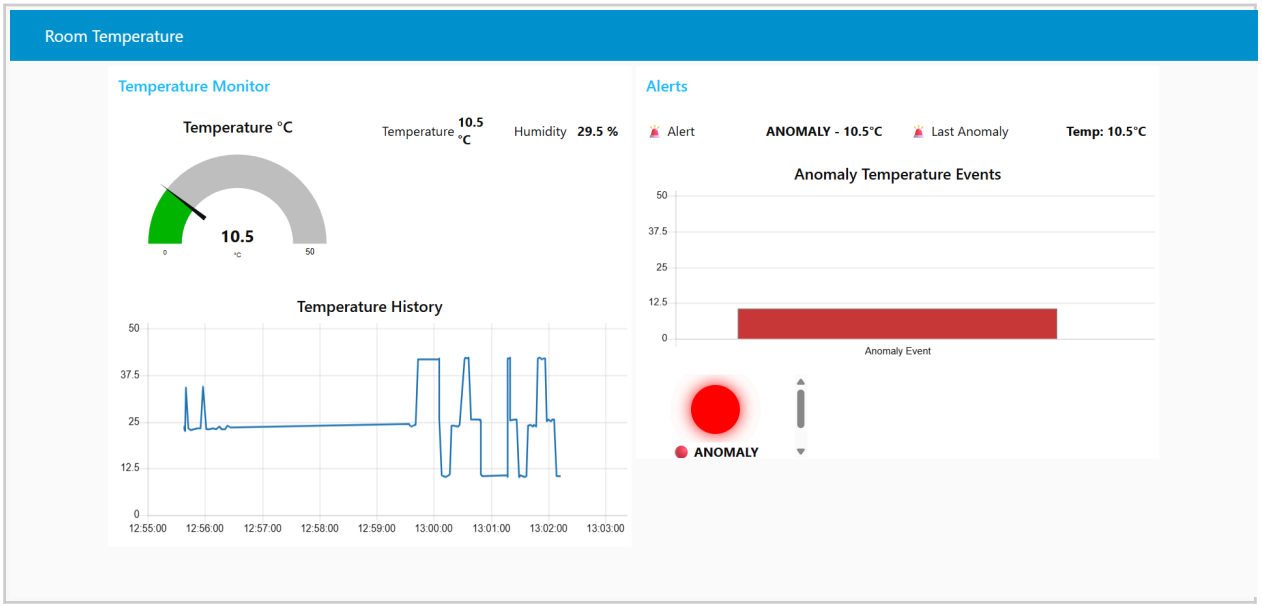

Figure 7: Node-RED Dashboard — Live Temperature Gauge, Chart, and Alert Panel

5. Data Simulation

In addition to real hardware operation, a Python simulation script (simulate.py) was developed to generate realistic synthetic temperature data for testing the full pipeline without physical hardware.

The simulation generates data with the following characteristics:

- Periodic variation - a sinusoidal base temperature ($26 + 3 \cdot \sin(t/20)$) mimics natural daily temperature cycles.
- Gaussian noise - random variation of ±0.5°C simulates sensor measurement noise.
- Injected anomalies - with 5% probability per reading, a spike of 8–15°C is added to simulate overheating events.

This approach allows full end-to-end testing of the MQTT pipeline, anomaly detection logic, and dashboard visualization before hardware integration, and also serves as a fallback demonstration mode.

```
PS C:\e20-co326-Room-Temperature-Anomaly-Detection\python> python simulate.py
C:\e20-co326-Room-Temperature-Anomaly-Detection\python\simulate.py:7: DeprecationWarning: Callback API version 1
  is deprecated, update to latest version
  client = mqtt.Client()
🚀 Simulation started! Sending data every 3 seconds...
📤 Sent: {"temperature": 26.0, "humidity": 48.0, "timestamp": 1777620059.9627833}
📤 Sent: {"temperature": 26.3, "humidity": 47.9, "timestamp": 1777620062.9658384}
📤 Sent: {"temperature": 26.4, "humidity": 49.3, "timestamp": 1777620066.0486405}
📤 Sent: {"temperature": 26.8, "humidity": 52.4, "timestamp": 1777620070.248372}
📤 Sent: {"temperature": 27.0, "humidity": 51.9, "timestamp": 1777620073.256995}
💥 Anomaly injected!
📤 Sent: {"temperature": 40.7, "humidity": 48.8, "timestamp": 1777620076.258062}
📤 Sent: {"temperature": 26.5, "humidity": 46.1, "timestamp": 1777620079.2598093}
💥 Anomaly injected!
📤 Sent: {"temperature": 40.1, "humidity": 54.5, "timestamp": 1777620082.2653193}
```

Figure 8: Simulation Output — Terminal Showing Injected Anomaly Spikes

6. Testing and Results

6.1 Hardware Verification

Hardware testing was conducted in stages to verify each component independently before full integration:

- DHT22 sensor test - a dedicated test sketch verified correct readings over the serial monitor, confirming GPIO4 wiring and pull-up resistor.
- Relay test - relay switching was verified independently, confirming GPIO5 trigger and audible buzzer activation.
- WiFi and MQTT connectivity - serial monitor confirmed successful WiFi association and MQTT broker connection.

6.2 Pipeline Integration Test

The complete pipeline was tested with the following sequence:

- docker-compose up -d - started MQTT broker and Node-RED.
- python edge_ai.py - started anomaly detection subscriber.
- ESP32 powered on - connected to WiFi and began publishing sensor data.
- Dashboard verified - gauge, chart, and text displays updated in real time.
- Anomaly triggered - hand-warming the DHT22 to above 32°C triggered ANOMALY status, activating the buzzer and red LED, and displaying the alert on the dashboard.

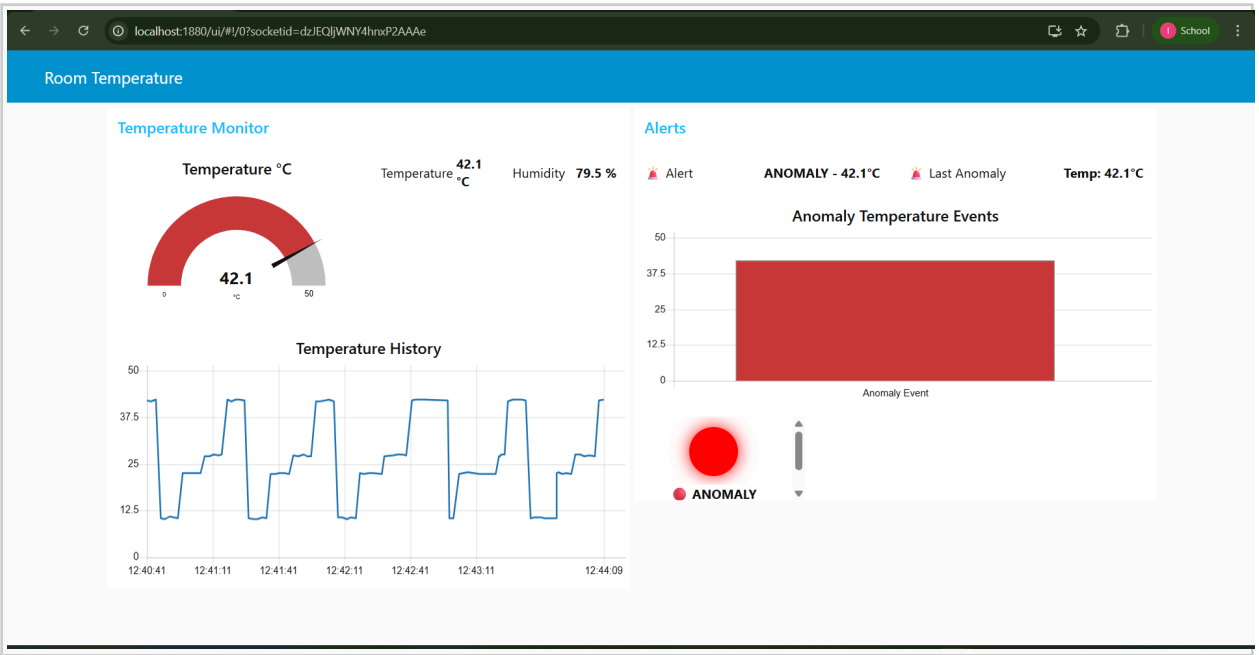

Figure 9: Dashboard During Anomaly Event — Red Alert Status and Gauge in Red Zone

6.3 Results Summary

Test	Result	Notes
DHT22 sensor reading	Pass	Accurate to ±0.5°C
WiFi connection	Pass	Connected to local network
MQTT publish from ESP32	Pass	Data visible in Node-RED debug
Edge AI anomaly detection	Pass	Spike and threshold rules triggered correctly
Dashboard live update	Pass	Gauge and chart update every 2–3 seconds
Alert display on dashboard	Pass	Alert panel shows anomaly reasons
Relay/buzzer activation	Pass	Activated within 1 message of anomaly
Simulation mode	Pass	Full pipeline functional without hardware

Table 5: Test Results Summary

7. Challenges and Solutions

Challenge	Root Cause	Solution
ESP32 could not reach MQTT broker	Docker bound to localhost only; WSL virtual IP not reachable from WiFi	Changed mosquitto.conf to bind 0.0.0.0; added Windows Firewall rule for port 1883; used WiFi IPv4 address in ESP32 firmware
Node-RED gauge showing raw JSON	MQTT input node passed full JSON object to gauge instead of extracted temperature value	Added function node to parse JSON and route individual fields to each widget
pip not working on Windows	Python installation had broken pip due to permissions	Re-ran ensurepip with Administrator CMD and --user flag
Docker images failed to download	DNS resolution failure on university network	Added 8.8.8.8 DNS to Docker Engine configuration; switched to mobile hotspot
Git commit failed due to node_modules	node-red/node_modules contained files with invalid Windows path characters	Added node-red/node_modules/ and node-red/.npm/ to .gitignore; removed cached tracking with git rm -r --cached
Two COM ports on ESP32-S3	ESP32-S3 has separate USB-CDC (upload) and UART (monitor) ports	Used COM7 for upload and COM8 for serial monitor in platformio.ini

Table 6: Challenges and Solutions

8. GitHub Repository Structure

The complete project is maintained on GitHub with continuous commits throughout development. The repository structure follows the module requirements:

```
e20-co326-Room-Temperature-Anomaly-Detection/
├── docker-compose.yml        ← Docker services (MQTT + Node-RED)
├── mosquitto/
│   └── mosquitto.conf        ← MQTT broker configuration
├── node-red/
│   └── flows.json            ← Node-RED dashboard flow
├── python/
│   ├── edge_ai.py            ← Anomaly detection AI
│   └── simulate.py           ← Sensor data simulation
├── room-temp-monitor/        ← PlatformIO ESP32-S3 firmware
│   ├── src/main.cpp          ← Main firmware code
│   └── platformio.ini        ← Board and library configuration
├── docs/
│   └── report.pdf            ← report
├── .gitignore
└── README.md
```

9. Conclusion

This project successfully demonstrated a complete Edge AI IoT system for room temperature anomaly detection. The system integrates a physical sensor layer (DHT22 + ESP32-S3), a communication layer (MQTT over WiFi), an edge intelligence layer (Python anomaly detection), and a visualization layer (Node-RED dashboard), all deployed using Docker for reproducibility.

The edge AI component runs entirely on the local PC, demonstrating the core principle of edge computing - processing data close to the source rather than relying on cloud infrastructure. This approach reduces latency, eliminates cloud dependency, and makes the system suitable for real industrial environments with limited connectivity.

The anomaly detection logic correctly identifies temperature spikes, sustained threshold violations, and drift patterns, with physical feedback provided through the relay and buzzer. The Node-RED dashboard provides clear real-time visualization for operators.

9.1 Future Improvements

- Implement a trained machine learning model (Isolation Forest or LSTM) to replace rule-based thresholds for more adaptive anomaly detection.
- Add InfluxDB and Grafana for historical data storage and long-term trend analysis.
- Deploy edge_ai.py inside Docker for a fully containerized, portable system.
- Add email or SMS notification for remote alerts.
- Expand to multi-room monitoring with multiple ESP32 nodes publishing to different MQTT topics.
- Implement MQTT authentication and TLS encryption for production security.

References

[1] Espressif Systems, ESP32-S3 Technical Reference Manual, 2023.

[2] Eclipse Foundation, Eclipse Mosquitto MQTT Broker Documentation, https://mosquitto.org/documentation/

[3] Node-RED Project, Node-RED Documentation, https://nodered.org/docs/

[4] beegee-tokyo, DHT sensor library for ESPx, https://github.com/beegee-tokyo/DHTesp

[5] Nick O'Leary, PubSubClient MQTT Library, https://pubsubclient.knolleary.net/

[6] Anthropic, Claude AI - used for development guidance and debugging assistance.